

实验5 多态

说明

一、实验目的

二、实验准备

三、实验内容

1. 实验任务1

2. 实验任务2

3. 实验任务3

4. 实验任务4

5. 实验任务5

四、实验结论

1. 实验任务1

2. 实验任务2

3. 实验任务3

4. 实验任务4

5. 实验任务5

五、实验总结(选)

六、实验文档提交要求

七、博客园编辑器使用补充说明

说明

• 阅读文档

请仔细阅读，明确任务要求、提交方式与截止时间。

• 实验任务说明

◦ 任务1-3，是**验证性**实验。读懂、运行、理解代码，掌握多态与相关 C++ 用法。

◦ 任务4-5，是**设计性**实验。参照验证性任务，模仿并完成设计、编码。

• 实践建议

◦ 每道练习，先独立思考，再讨论或查工具；缺少“亲手敲→犯错→调试→改错”环节，训练效果甚微。

◦ 随文档提供了部分代码。可下载、解压缩后使用。

如果你不熟悉C++语法，请手动敲入代码、刻意练习。

如有疑问，实验课请及时与老师或同学讨论。

一、实验目的

- 知道什么是类模板，会正确定义和实例化
- 理解运算符重载机制，会编写运算符函数，理解编译器如何将表达式转换为对运算符函数的调用
- 知道什么是抽象类，会正确定义和使用
- 基于问题场景，能合理使用继承、虚函数、抽象类实现接口继承与运行时多态

二、实验准备

系统浏览/复习以下教材章节：

- 继承：解决的问题场景、定义和用法（第7章）
- 多态：概念、分类、典型应用场景和用法（第8章）
- 类模板（9.1.2节）

三、实验内容

1. 实验任务1

验证性实验。综合应用组合、继承、多态实现出版物的分类管理。运行、理解代码，回答问题。

- 问题场景描述

模拟出版物的分类管理。抽象设计后，继承关系如下：



- 代码组织
 - publisher.hpp 类 `Publisher` 及其派生类 `Book` , `Film` , `Music` 声明
 - publisher.cpp 类 `Publisher` 及其派生类 `Book` , `Film` , `Music` 实现
 - task1.cpp 测试模块 + main

publisher.hpp

```
1  #pragma once
2
3  #include <string>
4
5  // 发行/出版物类: Publisher （抽象类）
6  class Publisher {
7  public:
8      Publisher(const std::string &name_ = "");           // 构造函数
9      virtual ~Publisher() = default;
10
11  public:
12      virtual void publish() const = 0;                   // 纯虚函数，作为接口继承
13      virtual void use() const = 0;                       // 纯虚函数，作为接口继承
14
15  protected:
16      std::string name;    // 发行/出版物名称
17  };
18
19  // 图书类: Book
```

```

20 class Book: public Publisher {
21 public:
22     Book(const std::string &name_ = "", const std::string &author_ = ""); // 构造函数
23
24 public:
25     void publish() const override; // 接口
26     void use() const override; // 接口
27
28 private:
29     std::string author; // 作者
30 };
31
32 // 电影类: Film
33 class Film: public Publisher {
34 public:
35     Film(const std::string &name_ = "", const std::string &director_ = ""); // 构造函数
36
37 public:
38     void publish() const override; // 接口
39     void use() const override; // 接口
40
41 private:
42     std::string director; // 导演
43 };
44
45
46 // 音乐类: Music
47 class Music: public Publisher {
48 public:
49     Music(const std::string &name_ = "", const std::string &artist_ = "");
50
51 public:
52     void publish() const override; // 接口
53     void use() const override; // 接口
54
55 private:
56     std::string artist; // 音乐艺术家名称
57 };

```

publisher.cpp

```

1  #include <iostream>
2  #include <string>
3  #include "publisher.hpp"
4
5  // Publisher类: 实现
6  Publisher::Publisher(const std::string &name_): name {name_} {
7  }
8
9
10 // Book类: 实现

```

```

11 Book::Book(const std::string &name_ , const std::string &author_ ): Publisher{name_},
    author{author_} {
12 }
13
14 void Book::publish() const {
15     std::cout << "Publishing book 《" << name << "》 by " << author << '\n';
16 }
17
18 void Book::use() const {
19     std::cout << "Reading book 《" << name << "》 by " << author << '\n';
20 }
21
22
23 // Film类: 实现
24 Film::Film(const std::string &name_, const std::string
    &director_):Publisher{name_},director{director_} {
25 }
26
27 void Film::publish() const {
28     std::cout << "Publishing film <" << name << "> directed by " << director << '\n';
29 }
30
31 void Film::use() const {
32     std::cout << "Watching film <" << name << "> directed by " << director << '\n';
33 }
34
35
36 // Music类: 实现
37 Music::Music(const std::string &name_, const std::string &artist_): Publisher{name_},
    artist{artist_} {
38 }
39
40 void Music::publish() const {
41     std::cout << "Publishing music <" << name << "> by " << artist << '\n';
42 }
43
44 void Music::use() const {
45     std::cout << "Listening to music <" << name << "> by " << artist << '\n';
46 }

```

task1.cpp

```

1 #include <memory>
2 #include <iostream>
3 #include <vector>
4 #include "publisher.hpp"
5
6 void test1() {
7     std::vector<Publisher *> v;
8
9     v.push_back(new Book("Harry Potter", "J.K. Rowling"));
10    v.push_back(new Film("The Godfather", "Francis Ford Coppola"));
11    v.push_back(new Music("Blowing in the wind", "Bob Dylan"));

```

```

12
13     for(Publisher *ptr: v) {
14         ptr->publish();
15         ptr->use();
16         std::cout << '\n';
17         delete ptr;
18     }
19 }
20
21 void test2() {
22     std::vector<std::unique_ptr<Publisher>> v;
23
24     v.push_back(std::make_unique<Book>("Harry Potter", "J.K. Rowling"));
25     v.push_back(std::make_unique<Film>("The Godfather", "Francis Ford Coppola"));
26     v.push_back(std::make_unique<Music>("Blowing in the wind", "Bob Dylan"));
27
28     for(const auto &ptr: v) {
29         ptr->publish();
30         ptr->use();
31         std::cout << '\n';
32     }
33 }
34
35 void test3() {
36     Book book("A Philosophy of Software Design", "John Ousterhout");
37     book.publish();
38     book.use();
39 }
40
41 int main() {
42     std::cout << "运行时多态：纯虚函数、抽象类\n";
43
44     std::cout << "\n测试1：使用原始指针\n";
45     test1();
46
47     std::cout << "\n测试2：使用智能指针\n";
48     test2();
49
50     std::cout << "\n测试3：直接使用类\n";
51     test3();
52 }

```

编译、运行程序，结合运行结果，理解代码并回答问题。

问题1：抽象类机制

- (1) 是什么决定了 `Publisher` 是抽象类？用一句话说明，并指出代码中的具体依据。
- (2) 如果在 `main.cpp` 里直接写 `Publisher p;` 能否编译通过？为什么？

问题2：纯虚函数与接口继承

- (1) `Book`、`Film`、`Music` 必须实现哪两个函数才能通过编译？请写出其完整函数声明。

(2) 在 publisher.cpp 的 `Film` 类实现中, 把两个成员函数实现里的 `const` 去掉 (保持函数体不变), 重新编译, 报错信息是什么?

问题3: 运行时多态与虚析构

(1) 在 `test1()` 里, `for (Publisher *ptr : v)` 中 `ptr` 的声明类型是什么?

(2) 当循环执行到 `ptr->publish();` 时, `ptr` 实际指向的对象类型分别有哪些? (按循环顺序写出)

(3) 基类 `Publisher` 的析构函数为何声明为 `virtual`? 若删除 `virtual`, 执行 `delete ptr;` 会出现什么问题?

2. 实验任务2

验证性实验。综合应用运算符重载、组合、标准库实现图书销售统计。

- 问题场景描述

模拟出版行业图书销售统计, 按指定关键字做销量统计、排序。

- 代码组织

- book.hpp 图书描述信息类Book声明
- book.cpp 图书描述信息类Book实现
- booksale.hpp 图书销售记录类BookSale声明
- booksale.cpp 图书销售记录类BookSale实现
- task2.cpp 测试模块 + main

book.hpp

```
1  #pragma once
2  #include <string>
3
4  // 图书描述信息类Book: 声明
5  class Book {
6  public:
7      Book(const std::string &name_,
8           const std::string &author_,
9           const std::string &translator_,
10          const std::string &isbn_,
11          double price_);
12
13      friend std::ostream& operator<<(std::ostream &out, const Book &book);
14
15  private:
16      std::string name;           // 书名
17      std::string author;        // 作者
18      std::string translator;    // 译者
19      std::string isbn;          // isbn号
20      double price;              // 定价
21  };
```

book.cpp

```

1  #include <iomanip>
2  #include <iostream>
3  #include <string>
4  #include "book.hpp"
5
6
7  // 图书描述信息类Book: 实现
8  Book::Book(const std::string &name_,
9             const std::string &author_,
10             const std::string &translator_,
11             const std::string &isbn_,
12             double price_):name{name_}, author{author_}, translator{translator_},
13             isbn{isbn_}, price{price_} {
14
15  // 运算符<<重载实现
16  std::ostream& operator<<(std::ostream &out, const Book &book) {
17      using std::left;
18      using std::setw;
19
20      out << left;
21      out << setw(15) << "书名:" << book.name << '\n'
22          << setw(15) << "作者:" << book.author << '\n'
23          << setw(15) << "译者:" << book.translator << '\n'
24          << setw(15) << "ISBN:" << book.isbn << '\n'
25          << setw(15) << "定价:" << book.price;
26
27      return out;
28  }

```

booksale.hpp

```

1  #pragma once
2
3  #include <string>
4  #include "book.hpp"
5
6  // 图书销售记录类BookSales: 声明
7  class BookSale {
8  public:
9      BookSale(const Book &rb_, double sales_price_, int sales_amount_);
10     int get_amount() const;    // 返回销售数量
11     double get_revenue() const; // 返回营收
12
13     friend std::ostream& operator<<(std::ostream &out, const BookSale &item);
14
15 private:
16     Book rb;
17     double sales_price;    // 售价
18     int sales_amount;      // 销售数量
19 };

```

booksale.cpp

```

1  #include <iomanip>
2  #include <iostream>
3  #include <string>
4  #include "booksale.hpp"
5
6  // 图书销售记录类BookSales: 实现
7  BookSale::BookSale(const Book &rb_,
8                      double sales_price_,
9                      int sales_amount_): rb{rb_}, sales_price{sales_price_},
10 sales_amount{sales_amount_} {
11 }
12
13 int BookSale::get_amount() const {
14     return sales_amount;
15 }
16
17 double BookSale::get_revenue() const {
18     return sales_amount * sales_price;
19 }
20
21 // 运算符<<重载实现
22 std::ostream& operator<<(std::ostream &out, const BookSale &item) {
23     using std::left;
24     using std::setw;
25
26     out << left;
27     out << item.rb << '\n'
28         << setw(15) << "售价:" << item.sales_price << '\n'
29         << setw(15) << "销售数量:" << item.sales_amount << '\n'
30         << setw(15) << "营收:" << item.get_revenue();
31
32     return out;
33 }

```

task2.cpp

```

1  #include "booksale.hpp"
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  #include <algorithm>
6
7  // 按图书销售数额比较
8  bool compare_by_amount(const BookSale &x1, const BookSale &x2) {
9     return x1.get_amount() > x2.get_amount();
10 }
11
12 void test() {
13     using namespace std;
14

```



```

15     vector<BookSale> sales_lst;           // 存放图书销售记录
16
17     int books_number;
18     cout << "录入图书数量: ";
19     cin >> books_number;
20
21     cout << "录入图书销售记录" << endl;
22     for(int i = 0; i < books_number; ++i) {
23         string name, author, translator, isbn;
24         float price;
25         cout << string(20, '-') << "第" << i+1 << "本图书信息录入" << string(20, '-') <<
endl;
26         cout << "录入书名: "; cin >> name;
27         cout << "录入作者: "; cin >> author;
28         cout << "录入译者: "; cin >> translator;
29         cout << "录入isbn: "; cin >> isbn;
30         cout << "录入定价: "; cin >> price;
31
32         Book book(name, author, translator, isbn, price);
33
34         float sales_price;
35         int sales_amount;
36
37         cout << "录入售价: "; cin >> sales_price;
38         cout << "录入销售数量: "; cin >> sales_amount;
39
40         BookSale record(book, sales_price, sales_amount);
41         sales_lst.push_back(record);
42     }
43
44     // 按销售册数排序
45     sort(sales_lst.begin(), sales_lst.end(), compare_by_amount);
46
47     // 按销售册数降序输出图书销售信息
48     cout << string(20, '=') << "图书销售统计" << string(20, '=') << endl;
49     for(auto &t: sales_lst) {
50         cout << t << endl;
51         cout << string(40, '-') << endl;
52     }
53 }
54
55 int main() {
56     test();
57 }

```

程序正确录入、编译后，测试效果如下：

```

录入图书数量：2
录入图书销售记录
-----第1本图书信息录入-----
录入书名：写作这回事
录入作者：斯蒂芬.金
录入译者：张坤
录入isbn：978-7-5321-5362-6
录入定价：29.00
录入售价：14.5
录入销售数量：15
-----第2本图书信息录入-----
录入书名：华氏451
录入作者：雷.布拉德伯里
录入译者：于而彦
录入isbn：978-7-5327-7549-1
录入定价：45.00
录入售价：32.00
录入销售数量：31
=====图书销售统计=====
书名：          华氏451
作者：          雷.布拉德伯里
译者：          于而彦
ISBN：          978-7-5327-7549-1
定价：          45
售价：          32
销售数量：      31
营收：          992
-----
书名：          写作这回事
作者：          斯蒂芬.金
译者：          张坤
ISBN：          978-7-5321-5362-6
定价：          29
售价：          14.5
销售数量：      15
营收：          217.5
-----

```

编译、运行程序，结合运行结果，理解代码并回答问题。

问题1：重载运算符<<

- (1) 找出运算符 << 被重载了几处？分别用于什么类型？
- (2) 找出使用重载 << 输出对象的代码，写在下面。

问题2：图书销售统计

- (1) 图书销售记录"按销售数量降序排序"，代码是如何实现的？
- (2) 拓展（选答*）：如果使用lambda表达式，如何实现"按销售数量降序排序"？

3. 实验任务3

验证性实验：类模板定义和使用。阅读、理解代码，结合运行回答问题。

- 问题场景描述

类A和类B除数据成员类型不同，其它都相同。类定义存在相似性

把类型参数化，让类的抽象设计更通用。

- 代码组织
 - task3_1.cpp 类A定义 + 类B定义 + 测试模块 + main
 - task3_2.cpp 类模板X定义 + 测试模块 + main

task3_1.cpp

```
1  #include <iostream>
2
3  // 类A的定义
4  class A {
5  public:
6      A(int x0, int y0);
7      void display() const;
8
9  private:
10     int x, y;
11 };
12
13 A::A(int x0, int y0): x{x0}, y{y0} {
14 }
15
16 void A::display() const {
17     std::cout << x << ", " << y << '\n';
18 }
19
20 // 类B的定义
21 class B {
22 public:
23     B(double x0, double y0);
24     void display() const;
25
26 private:
27     double x, y;
28 };
29
30 B::B(double x0, double y0): x{x0}, y{y0} {
31 }
32
33 void B::display() const {
34     std::cout << x << ", " << y << '\n';
35 }
36
37 void test() {
38     std::cout << "测试类A: " << '\n';
39     A a(3, 4);
40     a.display();
41
42     std::cout << "\n测试类B: " << '\n';
43     B b(3.2, 5.6);
44     b.display();
45 }
46
```

```
47 int main() {
48     test();
49 }
```

运行测试结果：

```
测试类A:
3, 4
测试类B:
3.2, 5.6
```

说明：

观察代码可见，类 A 与类 B 仅数据类型不同，其余代码完全一致。

类模板可将这些“相同部分”抽象成一份定义，把类型作为参数，实现通用化。

task3_2.cpp

```
1  #include <iostream>
2  #include <string>
3
4  // 定义类模板
5  template<typename T>
6  class X{
7  public:
8      X(T x0, T y0);
9      void display();
10
11 private:
12     T x, y;
13 };
14
15 template<typename T>
16 X<T>::X(T x0, T y0): x{x0}, y{y0} {
17 }
18
19 template<typename T>
20 void X<T>::display() {
21     std::cout << x << ", " << y << '\n';
22 }
23
24
25 void test() {
26     std::cout << "测试1: 用int实例化类模板X" << '\n';
27     X<int> x1(3, 4);
28     x1.display();
29
30
31     std::cout << "\n测试2: 用double实例化类模板X" << '\n';
32     X<double> x2(3.2, 5.6);
33     x2.display();
```

```

34
35     std::cout << "\n测试3: 用string实例化类模板X" << '\n';
36     X<std::string> x3("hello", "oop");
37     x3.display();
38 }
39
40 int main() {
41     test();
42 }

```

运行测试结果：

```

测试1: 用int实例化类模板X
3, 4

测试2: 用double实例化类模板X
3.2, 5.6

测试3: 用string实例化类模板X
hello, oop

```

说明：

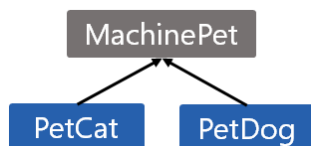
- 类模板用 `template<typename T>` 把“类型”抽象成参数，一份代码可生成多个具体类。
- 在类外实现成员函数时，必须加模板头并写完整类名 `X<T>`。
- 使用时要先“实例化”：`X<int> x1(3,4);`，编译器据此生成真正的类。
- 实例化类型必须支持模板内所有操作（如 `<<`），否则编译报错。
- 理解类模板后，再看标准库用法——`complex<double>`、`vector<int>`——只是现成模板的实例化而已。

4. 实验任务4

设计性实验。综合应用继承、多态，模拟简单机器宠物。

- 问题场景描述

模拟机器宠物，抽象后，继承关系如下：



- 定义抽象类：机器宠物类 `MachinePet`

每个机器宠物包含如下成员：

- 数据：昵称 (nickname)
- 接口要求

构造函数：用字符串初始化昵称

`get_nickname()`：供外部获取昵称

`talk()`: 返回叫声, 须支持运行时多态 (由派生类各自实现)。

- 定义电子宠物猫类 `PetCat`

公有继承自 `MachinePet`

- 无新增数据
- 构造函数: 用字符串初始化昵称
- 实现 `talk()` 返回猫叫声

- 定义电子宠物狗类 `PetDog`

公有继承自 `MachinePet`

- 无新增数据
- 构造函数: 用字符串初始化昵称
- 实现 `talk()` 返回狗叫声

- 代码组织

- `pet.hpp` 机器宠物抽象类 `MachinePet`、宠物猫类 `PetCat`、宠物狗类 `PetDog` 定义
- `task4.cpp` 测试模块 + `main`

测试代码`task4.cpp`已给出, 补足`pets.hpp`。

`task4.cpp`

```
1  #include <iostream>
2  #include <memory>
3  #include <vector>
4  #include "pet.hpp"
5
6  void test1() {
7      std::vector<MachinePet *> pets;
8
9      pets.push_back(new PetCat("miku"));
10     pets.push_back(new PetDog("da huang"));
11
12     for(MachinePet *ptr: pets) {
13         std::cout << ptr->get_nickname() << " says " << ptr->talk() << '\n';
14         delete ptr; // 须手动释放资源
15     }
16 }
17
18 void test2() {
19     std::vector<std::unique_ptr<MachinePet>> pets;
20
21     pets.push_back(std::make_unique<PetCat>("miku"));
22     pets.push_back(std::make_unique<PetDog>("da huang"));
23
24     for(auto const &ptr: pets)
25         std::cout << ptr->get_nickname() << " says " << ptr->talk() << '\n';
26 }
27
28 void test3() {
29     // MachinePet pet("little cutie"); // 编译报错: 无法定义抽象类对象
```

```

30
31     const PetCat cat("miku");
32     std::cout << cat.get_nickname() << " says " << cat.talk() << '\n';
33
34     const PetDog dog("da huang");
35     std::cout << dog.get_nickname() << " says " << dog.talk() << '\n';
36 }
37
38 int main() {
39     std::cout << "测试1: 使用原始指针\n";
40     test1();
41
42     std::cout << "\n测试2: 使用智能指针\n";
43     test2();
44
45     std::cout << "\n测试3: 直接使用类\n";
46     test3();
47 }

```

类定义正确补足后，预期测试效果如下：

```

测试1: 使用原始指针
miku says miao wu~
da huang says wang wang~

测试2: 使用智能指针
miku says miao wu~
da huang says wang wang~

测试3: 直接使用类
miku says miao wu~
da huang says wang wang~

```

Tips:

编写机器宠物抽象类 `MachinePet`、宠物猫类 `PetCat`、宠物狗类 `PetDog` 定义时，如不熟练，参照**任务1**。

注意点：

- 抽象类定义：若 test1 出现 `delete ptr;` 相关崩溃或**抽象类无法实例化**的报错，请复习**纯虚函数**和**虚析构**规则。
- 派生类构造函数初始化列表写法：昵称须提供给基类
- 拓展*：电子宠物猫/狗的 `talk()` 实现，如希望叫声效果拟真，可查阅网络，使用支持.midi/.wav等音频播放的工具函数

5. 实验任务5

设计性实验。综合应用运算符重载、类模板实现编译时多态。

- 问题场景描述

自定义简化版类模板 `Complex`, 实现类似C++标准库类模板 `complex`, 支持对类型的参数化。具体要求如下:

- 支持实例化类模板与各种构造

如 `Complex<double> c1; Complex<double> c2(1.0, 2.0); Complex<double> c3(c2);`

- 提供接口 `get_real()`, `get_image()` 返回实部和虚部

- 对类模板重载运算符, 支持如下操作: `c1 += c2`, `c1 + c2`, `c1 == c2`, `cin >> c1 >> c2`, `cout << c1 << c2`

- 代码组织

- `Complex.hpp` 类模板 `Complex` 定义

- `task5.cpp` 测试模块 + main

测试代码task5.cpp已给出。根据测试代码, 补足类模板 `Complex` 定义, 使代码运行后满足预期截图效果。

task5.cpp

```
1  #include <iostream>
2  #include "Complex.hpp"
3
4  void test1() {
5      using std::cout;
6      using std::boolalpha;
7
8      Complex<int> c1(2, -5), c2(c1);
9
10     cout << "c1 = " << c1 << '\n';
11     cout << "c2 = " << c2 << '\n';
12     cout << "c1 + c2 = " << c1 + c2 << '\n';
13
14     c1 += c2;
15     cout << "c1 = " << c1 << '\n';
16     cout << boolalpha << (c1 == c2) << '\n';
17 }
18
19 void test2() {
20     using std::cin;
21     using std::cout;
22
23     Complex<double> c1, c2;
24     cout << "Enter c1 and c2: ";
25     cin >> c1 >> c2;
26     cout << "c1 = " << c1 << '\n';
27     cout << "c2 = " << c2 << '\n';
28
29     const Complex<double> c3(c1);
30     cout << "c3.real = " << c3.get_real() << '\n';
```



```

31     cout << "c3.imag = " << c3.get_imag() << '\n';
32 }
33
34 int main() {
35     std::cout << "自定义类模板Complex测试1: \n";
36     test1();
37
38     std::cout << "\n自定义类模板Complex测试2: \n";
39     test2();
40 }

```

类模板 `Complex` 正确定义后，预期测试效果如下：

```

自定义类模板Complex测试1:
c1 = 2 - 5i
c2 = 2 - 5i
c1 + c2 = 4 - 10i
c1 = 4 - 10i
false

自定义类模板Complex测试2:
Enter c1 and c2: 1 -5 3 4
c1 = 1 - 5i
c2 = 3 + 4i
c3.real = 1
c3.imag = -5

```

Tips:

类模板定义和使用，可参照**任务3**。复数类及运算符重载，可参照学习通第8讲文档<动手手：实践体验-2>

四、实验结论

1. 实验任务1

此部分书写内容：

- 分别给出源码 `Publisher.hpp`, `Publisher.cpp`, `task1.cpp`，以及，运行测试截图
- 回答问题

问题1：抽象类机制

- (1) 是什么决定了 `Publisher` 是抽象类？用一句话说明，并指出代码中的具体依据。
- (2) 如果在 `main.cpp` 里直接写 `Publisher p;` 能否编译通过？为什么？

问题2：纯虚函数与接口继承

- (1) `Book`、`Film`、`Music` 必须实现哪两个函数才能通过编译？请写出其完整函数声明。
- (2) 在 `publisher.cpp` 的 `Film` 类实现中，把两个成员函数实现里的 `const` 去掉（保持函数体不变），重新编译，报错信息是什么？

问题3：运行时多态与虚析构

- (1) 在 `test1()` 里, `for (Publisher *ptr : v)` 中 `ptr` 的声明类型是什么?
- (2) 当循环执行到 `ptr->publish();` 时, `ptr` 实际指向的对象类型分别有哪些? (按循环顺序写出)
- (3) 基类 `Publisher` 的析构函数为何声明为 `virtual`? 若删除 `virtual`, 执行 `delete ptr;` 会出现什么问题?

2. 实验任务2

此部分书写内容:

- 分别给出 `book.hpp`, `book.cpp`, `booksale.hpp`, `booksale.cpp`, `task2.cpp` 源码, 以及, 运行测试截图 (要求换一组测试数据)
- 回答问题

问题1：重载运算符<<

- (1) 找出 `<<` 被重载了几处? 用于哪些类型?
- (2) 找出使用重载 `<<` 输出对象的代码, 写在下面。

问题2：图书销售统计

- (1) "按销售数量降序排序", 描述降序排序实现方式。
- (2) 拓展 (选答*) : 如果使用 `lambda` 表达式, 如何实现?

3. 实验任务3

验证性实验, 自行练习, 无需写入实验博客文档。

学习/理解此实验任务后, 参照此任务, 完成实验任务5简单类模板编程练习。

4. 实验任务4

此部分书写内容:

- 分别给出补足后的 `pet.hpp`、`task4.cpp` 源码, 以及, 运行测试截图

5. 实验任务5

此部分书写内容:

- 分别给出补足后的 `Complex.hpp`、`task5.cpp` 源码, 以及, 运行测试截图

五、实验总结(选)

此部分书写内容：（可以包括但不限于以下内容）

- 提炼本次实验的核心收获与思考
- 记录新发现、真实感受与待解疑问
- 任意想补充的反馈或分享

六、实验文档提交要求

- **提交形式**

请将实验文档以**在线技术博客**形式提交，发布后请在**相应实验作业**页面提交链接。

博客开通与提交方法详见：「[博客园博客开通及发布步骤_2024级_OOP.pdf](#)」

- **撰写内容**

必写：「四、实验结论」

选写：「五、实验总结」

要求：内容完整，表述简练，逻辑清晰，图文整洁

- **截止时间**

通常为实验课后一周内，具体**截止时间**以博客园发布的作业通知为准。（**提交时间**以博客园文章页面显示的日期为准）

截止后，欢迎通过[教学班级博客主页](#)相互评阅。

代码正确、风格规范、图文清晰、总结到位的博客，欢迎点赞推荐，一起进步！

七、博客园编辑器使用补充说明

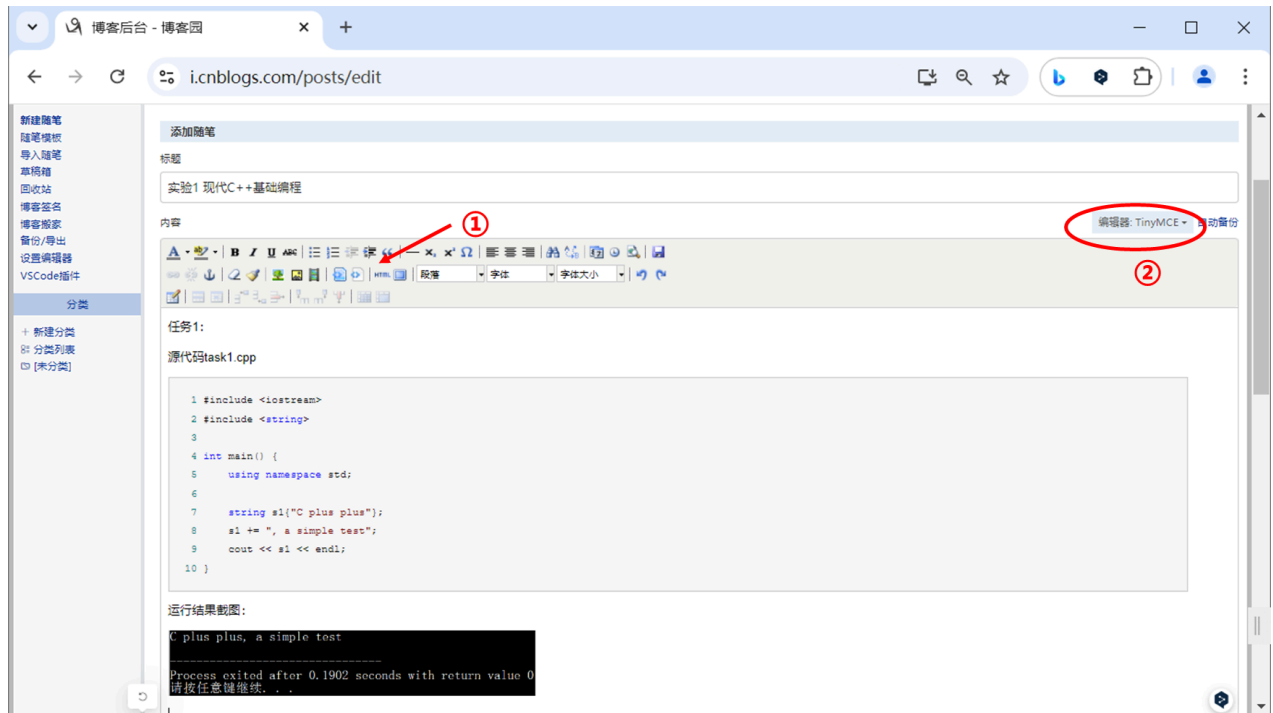
针对博客园平台使用的默认编辑器，插入代码和图片后，文字难以插入编辑的状况：

- 解决方案1

在插入代码和图片前，自己先编辑好文字，或者，用回车，预留好空白行

- 解决方案2

在编辑界面，点开页面对应的html源代码，在你想插入文字的地方加上html标签 `<p></p>`（图中①位置），然后再回到编辑界面，就会看到空白行。可以在那里编辑。



另，博客园平台设置页面支持更改默认编辑器（图中②位置）。目前，默认编辑器是markdown编辑器。

如果你暂不熟悉markdown基础用法，可切换成TinyMCE编辑器，其界面和使用风格接近office系列。

markdown编辑器使用参考：<https://www.runoob.com/markdown/md-tutorial.html>